LLM-as-Researcher: Comparative Analysis of AI Models for Autonomous Neural Network Training Optimization

Patrick Hughes
BMD PAT LLC
pat@bmdpat.com

March 23, 2026

Abstract

We evaluate large language models as autonomous ML researchers using the `autoresearch` framework [Karpathy, 2026]. Each model iteratively proposes modifications to a 50M-parameter language model's training code, runs 5-minute experiments on PubMed medical abstracts, and keeps or reverts changes based on validation bits-per-byte (val_bpb). In Phase 1 (exploratory), we sequentially ran Claude Sonnet 4, Claude Sonnet 4.6, and Claude Opus 4.6 (with 32K extended thinking), with each model building on previous improvements. Sonnet 4.6 achieved the highest keep rate (22.1%) with the lowest crash rate (8.7%), while Opus 4.6 found the deepest optimizations through multi-step causal reasoning. In Phase 2 (controlled, in progress), all models including GPT-4.1 start from an identical unoptimized baseline with $n = 3$ runs per model, enabling direct statistical comparison. We provide detailed failure analysis, cost-effectiveness data, and practical guidance for researchers implementing similar pipelines.

1 Introduction

Large language models have become standard coding assistants, but a more ambitious question remains open: can LLMs conduct research autonomously? Specifically, can they iteratively improve a machine learning training pipeline without human intervention, proposing hypotheses, running experiments, and learning from results?

Karpathy's `autoresearch` framework [Karpathy, 2026] operationalizes this question. An LLM agent is given a training script, a dataset, and a single objective metric. It proposes code modifications as JSON patches, the framework applies them, trains for a fixed duration, evaluates validation performance, and keeps the change only if it improves the metric. Otherwise the change is reverted and the agent is informed of the failure. The agent receives the full experiment history and tries again.

This paper makes the following contributions:

1. **Multi-model comparison.** We run three LLMs through sequential experimental conditions in Phase 1 and compare keep rates, crash rates, cost-effectiveness, and the quality of discovered optimizations. Phase 2 adds GPT-4.1 in a controlled head-to-head design with $n = 3$ replication.

2. **Extended thinking analysis.** We demonstrate that Claude Opus 4.6 with 32K extended thinking tokens finds multi-step optimization chains that shorter-context models miss, particularly on already-optimized baselines.

3. **Practical engineering insights.** We document Blackwell GPU (RTX 5070 Ti) compatibility issues, `torch.compile` failure modes, and cost planning guidance for researchers reproducing this work.

4. **Open data.** We report full experiment logs including every proposed change, its outcome, and the resulting val_bpb, enabling meta-analysis of LLM research strategies.

2 Related Work

Automated Machine Learning. AutoML [Hutter et al., 2019] encompasses hyperparameter optimization, neural architecture search (NAS) [Zoph and Le, 2017], and automated feature engineering. NAS methods typically define a fixed search space and use reinforcement learning or evolutionary algorithms to explore it. Our setting differs: the LLM operates on raw Python source code with no predefined search space, making the problem both harder (unbounded space) and more flexible (any valid code change is permitted).

LLM Code Generation. Codex [Chen et al., 2021] and AlphaCode demonstrated that LLMs can generate functionally correct programs from specifications. More recent models – Claude Sonnet 4.6, Opus 4.6, and GPT-4.1 – represent significant advances in code understanding and generation. However, generating code that *improves* an existing training pipeline requires understanding not just syntax but the interactions between optimizer hyperparameters, model architecture, and training dynamics.

AI-Driven Scientific Discovery. The AI Scientist [Lu et al., 2024] demonstrated end-to-end automated research including idea generation, experimentation, and paper writing. Our work is complementary but narrower in scope: we focus specifically on the optimization loop and use it as a controlled testbed for comparing LLM capabilities. Where The AI Scientist evaluates whether AI can do science, we evaluate which AI does it best.

The autoresearch Framework. Karpathy introduced `autoresearch` in early 2026 [Karpathy, 2026] as an open-source tool for autonomous training optimization. The framework provides the agent loop, experiment logging, and keep/revert logic. Our work extends the framework with adaptive temperature scheduling, enhanced failure categorization, and multi-model support including Azure OpenAI endpoints.

3 Methodology

3.1 Framework

The `autoresearch` loop proceeds as follows:

Algorithm 1 Autoresearch Optimization Loop

```
Initialize train.py with baseline configuration
Train baseline, record val_bpb_0
for i = 1 to N do
    Send experiment history + current train.py to LLM
    LLM returns JSON patch (search/replace pairs)
    Apply patch to train.py
    Train for 5 minutes, record val_bpb_i
    if val_bpb_i < val_bpb_best then
        Keep change, update val_bpb_best
    else
        Revert train.py to previous version
    end if
    Append result (kept/discarded/crashed) to history
end for
```

Several extensions were implemented for robustness:

- **Categorized history summaries.** The full experiment history is sent to the LLM each round, organized into kept, discarded, and crashed categories with near-miss highlighting for experiments that came close to improving.
- **Adaptive temperature.** For Sonnet models, the API temperature increases linearly from 0.0 to 0.5 during failure streaks, encouraging exploration when repeated proposals fail. Temperature resets to 0.0 after a successful keep.
- **FatalAPIError handling.** Billing failures, rate limits, and network errors are caught and retried with exponential backoff rather than counted as experiment failures.
- **Auto-restart.** `torch.compile` inductor bugs cause intermittent segfaults. The framework detects these and restarts the training process automatically.

3.2 Model Architecture

The target model is a 50M-parameter decoder-only transformer, configured as shown in Table 1.

Table 1: Baseline model architecture.

Parameter	**Value**
Parameters	50M
Layers	8
Attention heads	4
Embedding dimension	512
Optimizer	Muon + AdamW (embeddings)
Attention mechanism	SDPA
Compilation	`torch.compile`
Max sequence length	2048

The Muon optimizer [Jordan et al., 2024] is used for all matrix-shaped parameters, while AdamW handles embedding parameters. Flash Attention 3 [Dao, 2024] was unavailable on the Blackwell architecture at the time of experiments, so PyTorch's scaled dot-product attention (SDPA) was used as a fallback.

The use of `torch.compile` is important context for the experiment results: it constrains the space of valid code changes, since many architectural modifications break the compilation graph. This is a significant source of crashes, particularly for models with weaker code understanding.

3.3 Dataset

All experiments use PubMed medical abstracts from the HuggingFace dataset `uiyunkim-hub/pubmed-abstract`, containing approximately 27.7 million abstracts. A BPE tokenizer with vocabulary size 8,192 was trained on the corpus. Maximum sequence length was set to 2,048 tokens.

PubMed abstracts were chosen for several reasons: the text is relatively homogeneous (structured medical writing), the dataset is large enough to avoid overfitting in 5-minute training runs, and the medical domain provides a practical testbed for language modeling on specialized technical text.

The evaluation metric is validation bits-per-byte (val_bpb), which measures compression efficiency on held-out data. Lower values indicate better language modeling. This metric is preferred over perplexity because it is tokenizer-independent, enabling fair comparison across different vocabulary sizes if the tokenizer is modified.

3.4 Hardware

All experiments were conducted on a single workstation:

- **GPU:** NVIDIA GeForce RTX 5070 Ti, 16GB VRAM, Blackwell architecture (SM 12.0)
- **OS:** WSL2 Ubuntu 22.04 on Windows 11 Pro
- **Software:** Python 3.12, PyTorch 2.9.1+cu128, CUDA 12.8
- **Thermal management:** Training pauses at 80°C GPU temperature, aborts at 90°C

The RTX 5070 Ti was among the first consumer Blackwell GPUs available and presented several compatibility challenges documented in Section 6.3.

3.5 LLM Configurations

Three LLM configurations were used in Phase 1, summarized in Table 2.

Table 2: LLM configurations for each experimental run.

Model	API	Thinking	Temperature	Timeout	Est. Cost/Exp
Claude Sonnet 4	Anthropic	None	Adaptive (0–0.5)	180s	$0.06
Claude Sonnet 4.6	Anthropic	None	Adaptive (0–0.5)	180s	$0.06
Claude Opus 4.6	Anthropic	32K tokens	1.0 (API req.)	900s	$1–2

GPT-4.1 is evaluated in Phase 2 only (Section 5).

Opus 4.6 uses temperature 1.0, which is required by Anthropic's extended thinking API (not a tunable parameter). The 900-second timeout accommodates the longer response times when the model uses its full 32K thinking budget on complex prompts with long experiment histories. The substantially higher cost per experiment ($1–2 vs. $0.06) reflects the extended thinking token usage, which is billed as output tokens.

4 Phase 1: Exploratory Sequential Optimization

In Phase 1, models were run as a relay: Sonnet 4.6 optimized the training code from an unmodified baseline, then Opus 4.6 continued from Sonnet 4.6's best result. (Sonnet 4 ran separately on a different baseline as an earlier pilot.) This sequential design means each model faced a progressively harder optimization landscape – Opus 4.6 had to find improvements that Sonnet 4.6 could not, not compete on the same starting code. Phase 1 should therefore be read as a relay narrative (Figure 1), not a head-to-head competition. Phase 2 (Section 5) provides the controlled comparison.

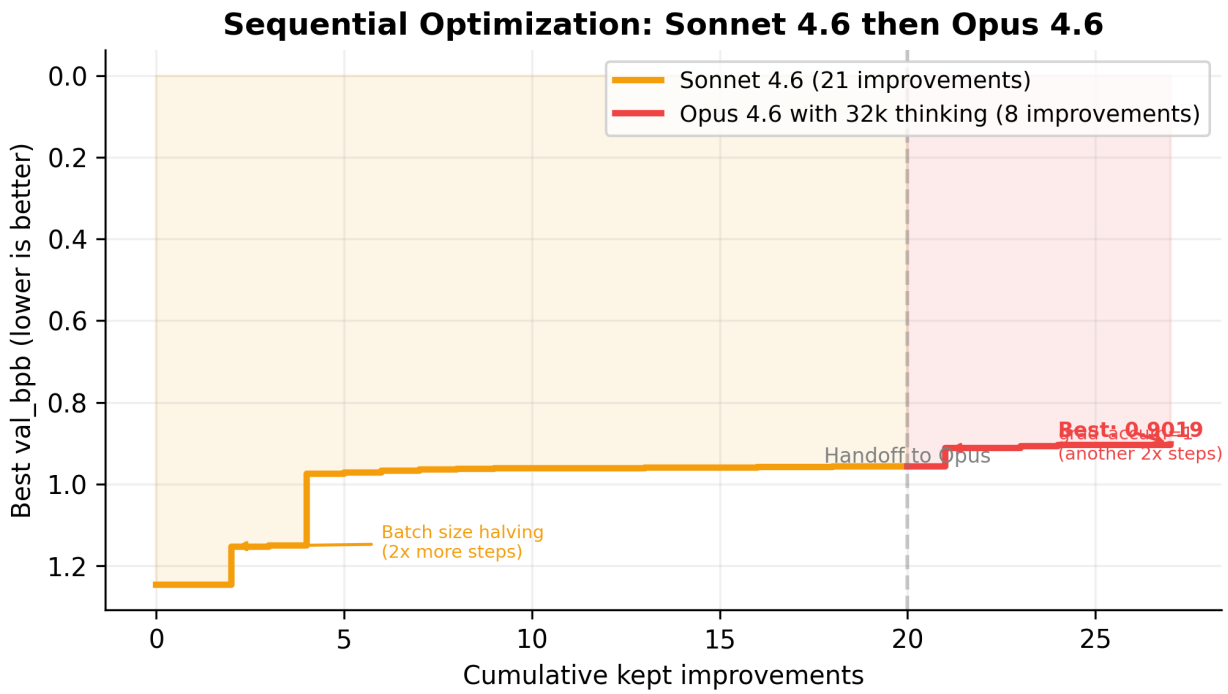

Figure 1: Phase 1 optimization relay. Sonnet 4.6 made 21 improvements from an unoptimized baseline (val_bpb 1.245 → 0.956), then Opus 4.6 with extended thinking continued from that optimized code and found 8 further improvements (0.956 → 0.902). Key discoveries are annotated. The x-axis shows cumulative kept improvements, not total experiments.

4.1 Overall Comparison

Table 3 summarizes the performance of each model across all experiments.

Table 3: Overall results for each LLM. Keep rate is computed over non-crashed experiments. Improvement is relative to each run's starting baseline.

Model	Exps	Kept (%)	Disc.	Crashed (%)	Baseline	Best val_bpb	Improv. (%)
Sonnet 4	147	5 (5.2%)	45	50 (34.0%)	0.9481	0.9362	1.25
Sonnet 4.6	104	21 (22.1%)	70	9 (8.7%)	1.2453	0.9559	23.24
Opus 4.6	111	8 (8.9%)	64	21 (18.9%)	0.9557	0.9019	5.64

Skipped experiments. Not all experiments reached the training stage. When an LLM returned a response that could not be parsed as a valid JSON code patch, the experiment was skipped with no training run. Sonnet 4 had 47 such skipped experiments (32% of total), Sonnet 4.6 had 4 (4%), and Opus 4.6 had 18 (16%). The skipped counts are included in the experiment totals but excluded from the kept/discarded/crashed breakdown. This metric is a practical measure of instruction-following reliability: Sonnet 4.6's near-zero skip rate reflects strong adherence to the required JSON output format, while Sonnet 4's high skip rate indicates frequent formatting failures that wasted experiment slots.

Important caveat on baselines. The three runs did not all start from the same baseline, which complicates direct comparison:

- **Sonnet 4** started from a partially optimized baseline (val_bpb = 0.948) produced by earlier manual tuning.
- **Sonnet 4.6** started from the clean, unmodified master branch (val_bpb = 1.245), giving it the most room for improvement.
- **Opus 4.6** started from Sonnet 4.6's best result (val_bpb = 0.956), an already well-optimized configuration.

As shown in Figure 1, Phase 1 is best understood as a relay: Sonnet 4.6 handled the initial optimization (finding big wins like batch size halving and warmdown tuning), then Opus 4.6 took over and found subtler improvements requiring multi-step reasoning. Comparing their keep rates or improvement percentages directly is misleading because they operated on fundamentally different difficulty levels. Phase 2 addresses this with a controlled identical-baseline design.

4.2 Key Discoveries by Model

4.2.1 Claude Sonnet 4

Sonnet 4 found 5 improvements over 147 experiments. The successful changes were:

1. **Total batch size halving** (`TOTAL_BATCH_SIZE` from 2^{17} to 2^{16}): More frequent gradient updates improved convergence within the 5-minute window.
2. **Warmdown ratio tuning** ($0.5 \rightarrow 0.75$): Extended the cosine decay phase of the learning rate schedule.
3. **Adaptive gradient clipping**: Replaced fixed gradient clipping with a norm-based adaptive scheme.

4. **Per-group weight decay adjustments**: Differentiated weight decay between Muon and AdamW parameter groups.

A notable failure mode emerged in later experiments: Sonnet 4 became stuck in a loop proposing minor variations of weight decay values, unable to explore qualitatively different optimization strategies. The 34% crash rate was driven by attempts to modify normalization layers, rotary position embeddings (RoPE), and exponential moving average (EMA) – all changes that broke `torch.compile`.

4.2.2 Claude Sonnet 4.6

Sonnet 4.6 was the most efficient optimizer, finding 21 improvements in 104 experiments. Its strategy was systematic:

1. **Phase 1 – Big wins.** Batch size halving and warmdown ratio tuning, similar to Sonnet 4's discoveries.
2. **Phase 2 – Optimizer tuning.** Systematic per-group hyperparameter adjustments: epsilon values, beta parameters, and learning rates for both Muon and AdamW groups.
3. **Phase 3 – Fine-grained search.** Small incremental adjustments to already-tuned parameters.

Sonnet 4.6's 8.7% crash rate was the lowest of all models, indicating strong understanding of `torch.compile` constraints. It rarely proposed architectural changes that would break the compilation graph, instead focusing on numerical hyperparameter modifications.

4.2.3 Claude Opus 4.6 (Extended Thinking)

Opus 4.6 with 32K extended thinking found 8 improvements in 111 experiments, but the nature of its discoveries was qualitatively different. Starting from an already well-optimized baseline (Sonnet 4.6's best), it needed to find deeper optimizations.

The key discovery chain illustrates the value of extended thinking:

1. **Gradient accumulation removal** (`grad_accum` $2 \rightarrow 1$): The model reasoned that with the already-halved batch size, removing gradient accumulation would double the number of optimizer steps within the 5-minute window.
2. **Weight decay compensation**: The model recognized that doubling optimizer steps meant doubling the effective weight decay applications per training run, and halved the weight decay to compensate.
3. **Matrix learning rate increase**: With more frequent but smaller steps, the model increased the Muon learning rate for matrix parameters.
4. **Exponential `x0_lambda` gradient**: Introduced exponential scheduling for the initial parameter regularization term.
5. **Gradient clip floor adjustment**: Tightened the gradient clipping threshold after observing training stability.

This multi-step reasoning chain – understanding that one change creates side effects that must be compensated – is characteristic of the extended thinking capability. The model's thinking traces (visible in the API response) showed explicit reasoning about second-order effects of each proposed change.

The 18.9% crash rate was higher than Sonnet 4.6's, reflecting more aggressive experimentation. Opus 4.6 attempted structural changes that Sonnet 4.6 avoided, accepting a higher crash rate in exchange for exploring a larger optimization space.

4.3 Failure Analysis

Figure 2 illustrates the crash rate distribution across models.

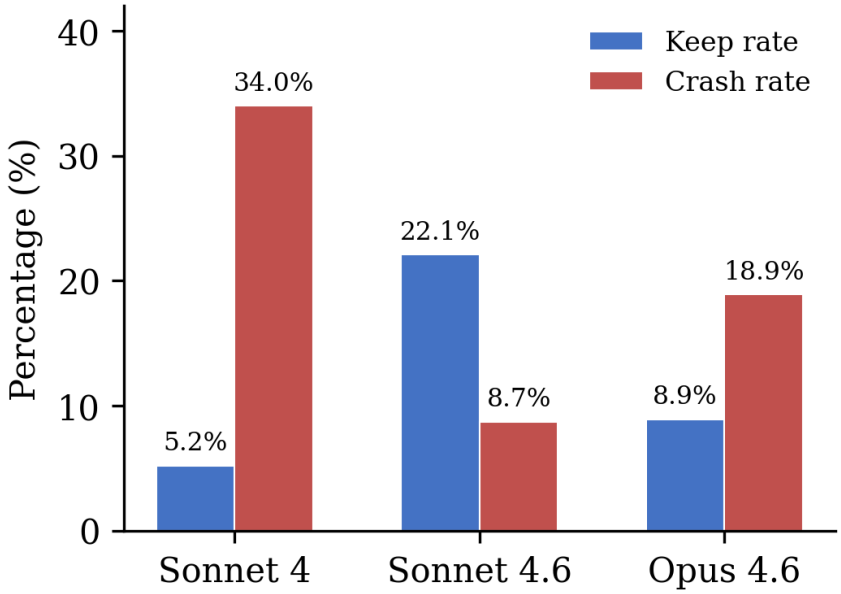

Figure 2: Keep rate (non-crash experiments that improved val_bpb) and crash rate by model. Sonnet 4.6 achieved the highest keep rate and lowest crash rate among the three Phase 1 models.

We categorize crashes into five types:

1. **Compilation failures**: Changes that break `torch.compile` graph tracing or cause GPU kernel launch errors. These include dynamic shapes, data-dependent branches, and unsupported operations.
2. **Runtime errors**: Out-of-memory (OOM) from increased batch or model sizes, numerical instability (NaN losses), and dimension mismatches.
3. **Inductor bugs**: Intermittent `torch.compile` backend failures unrelated to the proposed change.
4. **Apply failed**: The LLM's proposed code patch could not be applied due to ambiguous or missing match targets.
5. **Uncategorized**: Crashes where the error log was truncated, preventing automated classification.

Table 4 shows the breakdown of crash types by model.

Table 4: Crash categorization by model, based on error log analysis. "Compilation" includes `torch.compile` graph tracing failures and GPU kernel launch errors. "Apply failed" means the LLM's proposed code patch could not be applied (ambiguous or missing match). "Inductor" refers to intermittent PyTorch inductor backend bugs unrelated to the proposed change.

Crash Type	**Sonnet 4**	**Sonnet 4.6**	**Opus 4.6**
Compilation failure	15 (30%)	0 (0%)	11 (52%)
Runtime error	10 (20%)	1 (11%)	1 (5%)
Inductor bug	3 (6%)	1 (11%)	1 (5%)
Apply failed	1 (2%)	5 (56%)	0 (0%)
Uncategorized	21 (42%)	2 (22%)	8 (38%)
Total	50	9	21

Compilation failures and uncategorized crashes dominate. The high "uncategorized" count reflects crashes where the error message was truncated in the TSV log, making automated classification difficult. Sonnet 4.6's crashes were predominantly failed code patches (the LLM proposed changes that could not be applied to the source), rather than training failures – indicating conservative but imprecise code modification strategies.

4.4 Cost-Effectiveness

Table 5 presents cost-effectiveness metrics for each model.

Table 5: Cost-effectiveness analysis. API cost is estimated from token usage. GPU cost assumes \$0.50/hr equivalent for electricity and amortization. Cost per 1% improvement is the total cost divided by the percentage improvement achieved.

Metric	**Sonnet 4**	**Sonnet 4.6**	**Opus 4.6**
API cost (est.)	\$8.82	\$6.24	\$50.00
GPU time (hrs)	12.25	8.67	9.25
GPU cost (est.)	\$6.13	\$4.33	\$4.63
Total cost (est.)	\$14.95	\$10.57	\$54.63
Improvement (%)	1.25	23.24	5.64
Cost / 1% impr.	**\$11.96**	**\$0.45**	**\$9.69**

Sonnet 4.6 is the clear efficiency winner at \$0.45 per 1% improvement. However, this comparison is influenced by the baseline difference: Sonnet 4.6 started from the unoptimized master branch where improvements are easier to find. A fairer comparison would control for baseline difficulty, which we discuss in Section 6.4.

Opus 4.6's cost per 1% improvement (\$9.69) appears expensive, but it operated on a much harder baseline. The absolute best val_bpb of 0.9019 was only achievable through Opus 4.6's extended reasoning, making it cost-effective when measured against the final performance ceiling rather than percentage improvement.

5 Phase 2: Controlled Head-to-Head Comparison (In Progress)

To address the baseline inconsistency identified in Phase 1, we designed a controlled experiment where all models start from an identical, unoptimized baseline.

5.1 Experimental Design

The controlled comparison uses the following protocol:

1. **Frozen baseline.** We reset `train.py` to the upstream Karpathy defaults (Table 6), adding only the minimum hardware compatibility patches for Blackwell GPUs (SDPA attention fallback). No hyperparameter optimization is included.

2. **Identical starting point.** All models branch from the same git commit (`3fb6704`), ensuring byte-identical starting code.

3. $n = 3$ **replication.** Each model runs three independent 100-experiment sessions. This enables computation of mean and standard deviation for keep rate, crash rate, and final val_bpb, providing statistical confidence in the comparison.

4. **Same hardware and dataset.** All runs use the RTX 5070 Ti with PubMed abstracts, matching Phase 1 conditions.

Table 6: Controlled baseline hyperparameters (upstream Karpathy defaults).

Parameter	**Value**
WARMDOWN_RATIO	0.5
MATRIX_LR	0.04
WEIGHT_DECAY	0.2
EMBEDDING_LR	0.6
TOTAL_BATCH_SIZE	2^{19} (524K tokens)
FINAL_LR_FRAC	0.0
Baseline val_bpb	1.097

The unoptimized baseline (val_bpb = 1.097) provides substantial room for improvement. Known optimizations such as batch size reduction, warmdown ratio tuning, and per-group optimizer adjustments are all available for the models to discover independently.

5.2 Run Schedule

Table 7 summarizes the planned runs. GPT-4.1 runs use Azure credits at zero marginal cost. Sonnet runs use the Anthropic API. Opus runs are the most expensive due to the 32K extended thinking budget.

Table 7: Phase 2 controlled comparison: planned runs.

Run	Model	API	Status	Est. Cost
1–3	GPT-4.1	Azure OpenAI	In progress	$0
4–6	Claude Sonnet 4.6	Anthropic	Planned	$18
7–9	Claude Sonnet 4	Anthropic	Planned	$18
10–12	Claude Opus 4.6	Anthropic	Planned	$150
Total (12 runs × 100 experiments)				**$186**

5.3 Expected Analyses

With $n = 3$ runs per model, Phase 2 will enable:

- **Statistical comparison** of keep rates and crash rates with confidence intervals.
- **Reproducibility assessment** via variance across runs of the same model.
- **Learning curve analysis** comparing how quickly each model discovers key optimizations (e.g., batch size halving).
- **Strategy diversity analysis** comparing whether different runs of the same model discover the same or different optimization paths.

Phase 2 results will be reported in v2 of this manuscript.

6 Discussion

6.1 Model Capability Translates to Research Quality

A clear hierarchy emerges from the results:

1. **Constraint awareness** (crash avoidance): Sonnet 4.6 > Opus 4.6 > Sonnet 4
2. **Optimization efficiency** (keep rate): Sonnet 4.6 (22.1%) > Opus 4.6 (8.9%) > Sonnet 4 (5.2%)
3. **Depth of insight** (final val_bpb): Opus 4.6 (0.9019) > Sonnet 4 (0.9362) > Sonnet 4.6 (0.9559)

Constraint awareness – the ability to learn from failed experiments and avoid repeating the same categories of errors – appears to be the most differentiating factor for practical efficiency. Sonnet 4.6's low crash rate meant that almost every API call produced a meaningful result (either a kept improvement or an informative discard), while Sonnet 4 lost 34% of its compute budget to crashes.

The depth-of-insight ranking differs from the efficiency ranking because Opus 4.6 operated on a harder baseline. This suggests that different models may be optimal for different phases of optimization: Sonnet 4.6 for rapid initial improvement, Opus 4.6 for squeezing out final gains.

6.2 Extended Thinking for Hard Problems

The `grad_accum=1` discovery chain (Section 4.2) demonstrates why extended thinking matters. The reasoning required:

1. Recognizing that `grad_accum=2` with batch size 32 means the optimizer updates every 64 samples.
2. Reasoning that `grad_accum=1` would double optimizer updates in the same wall-clock time.
3. Predicting that doubled updates would effectively double weight decay application.
4. Proposing halved weight decay to compensate.
5. Further adjusting learning rates for the new update frequency.

Each step depends on the previous one. Without extended thinking, the model would need to discover each compensation independently through trial and error, requiring multiple experiment rounds. With extended thinking, Opus 4.6 proposed steps 1–4 as a single coherent change, which succeeded on the first attempt.

This pattern – multi-step causal reasoning about optimization dynamics – is precisely the type of problem where extended thinking provides the most value. For simpler optimizations (batch size tuning, learning rate adjustments), the additional reasoning budget is unnecessary and the cost premium is not justified.

6.3 Blackwell GPU Challenges

Running `autoresearch` on an RTX 5070 Ti (Blackwell, SM 12.0) required solving several compatibility issues that may affect other researchers with new hardware:

1. **`torch.compile` inductor bugs.** Intermittent segmentation faults during compilation, unrelated to the code being compiled. Our auto-restart mechanism handled these transparently, but they add noise to experiment timings. We observed roughly 1 inductor crash per 30 experiments.
2. **Jinja2 version sensitivity.** PyTorch 2.9.1 with CUDA 12.8 requires `jinja2 < 3.1.5` due to template rendering changes in the Triton backend. The symptom is a cryptic compilation error with no obvious connection to Jinja2.
3. **Python version requirements.** Python 3.10's `sre_parse` module has bugs triggered by certain regex patterns in the PyTorch codebase. Python 3.12 resolves these.
4. **Flash Attention unavailability.** Flash Attention 3 [Dao, 2024] had not been ported to Blackwell at the time of these experiments. SDPA provides the same functionality with slightly lower throughput.

6.4 Limitations

Several limitations constrain the generalizability of our findings:

1. **Single GPU, single dataset.** All experiments used one GPU configuration and one dataset. Results may differ on larger models, different hardware, or different text domains.

2. **Hyperparameter-focused search.** Due to `torch.compile` fragility, the effective search space was dominated by numerical hyperparameter changes. Architecture search results might differ, particularly on frameworks that support dynamic modifications more gracefully.

3. **val_bpb as a proxy metric.** Bits-per-byte measures language modeling quality but does not directly measure downstream task performance (e.g., medical information extraction from PubMed abstracts).

4. **Different baselines (Phase 1).** The sequential nature of Phase 1 experiments prevents apples-to-apples comparison. Phase 2 (Section 5) addresses this with a controlled identical-baseline design.

5. **Single runs (Phase 1).** Each model was run once in Phase 1. Phase 2 uses $n = 3$ runs per model to assess reproducibility and compute confidence intervals.

6. **API pricing changes.** Cost estimates reflect pricing at the time of experiments (March 2026) and will likely change.

7 Practical Guide for Reproduction

This section provides step-by-step instructions for researchers wishing to reproduce or extend our results. The `autoresearch` framework is available at https://github.com/karpathy/autoresearch.

7.1 Hardware Requirements

- **Minimum:** Any NVIDIA GPU with 12+ GB VRAM and CUDA 12.0+ support. The 50M-parameter model fits comfortably in 8GB, but training overhead requires additional headroom.
- **Recommended:** RTX 4070 Ti or newer. Older architectures (Ampere, Hopper) have more stable `torch.compile` support.
- **For Blackwell GPUs (50-series):** Expect intermittent inductor bugs. Ensure auto-restart is enabled in the framework.

7.2 Environment Setup

1. **Python 3.12+** is required. Do not use Python 3.10 (regex bugs) or 3.11 (untested).

2. **Pin Jinja2:** `pip install "jinja2<3.1.5"` before installing PyTorch.

3. **WSL2 recommended on Windows.** Native Windows PyTorch lacks full `torch.compile` support.

4. **API keys:** Set `ANTHROPIC_API_KEY` for Claude models and configure Azure OpenAI endpoint for GPT-4.1.

7.3 Running Experiments

1. Clone the repository and install dependencies.

2. Prepare the PubMed dataset: `python data/prepare_pubmed.py`

3. Run the baseline: `python train.py` (verify val_bpb matches expected range).

4. Launch autoresearch: `python autoresearch.py -model claude-sonnet-4.6 -experiments 100`

5. Monitor via the Rich TUI dashboard showing real-time experiment status, val_bpb history, and GPU metrics.

7.4 Common Pitfalls

- **GPU thermal throttling.** Consumer GPUs without adequate cooling will thermal-throttle during sustained training. Monitor temperatures. Our framework pauses at 80°C and aborts at 90°C to prevent hardware damage.

- **API billing surprises.** Opus 4.6 with extended thinking can cost $1–2 per experiment. Budget $50–100 for a 100-experiment Opus run. Set billing alerts.

- **Disk space.** Experiment logs and model checkpoints accumulate. Expect 5–10 GB per 100-experiment run.

- **`torch.compile` cache.** The compilation cache can grow large and occasionally become corrupted. Clear it (`rm -rf ~/.cache/torch/inductor`) if experiments show unexplained compilation failures.

7.5 Cost Planning

Table 8 provides budget estimates for various experiment scales.

Table 8: Estimated costs for different experiment configurations (API costs only, March 2026 pricing).

Model	50 Experiments	100 Experiments	200 Experiments
Sonnet 4 / 4.6	$3	$6	$12
Opus 4.6 (32K thinking)	$50–100	$100–200	$200–400

Our recommendation: start with 100 experiments of Sonnet 4.6 to establish a strong baseline (~$6 API cost), then run 50–100 experiments of Opus 4.6 on the optimized result if further improvement is desired (~$50–100 API cost).

8 Conclusion

We presented an experimental comparison of LLMs as autonomous ML researchers, evaluating three models in Phase 1 on the task of optimizing a 50M-parameter language model's training configuration. While Phase 1's sequential design limits direct comparison (addressed by the controlled Phase 2), several findings emerge:

1. **LLM capability directly impacts autonomous research quality.** Crash rates, keep rates, and the depth of discovered optimizations all correlate with model capability. The gap between the best (Sonnet 4.6) and worst (Sonnet 4) Phase 1 performers is substantial.

2. **Sonnet 4.6 is the efficiency optimum for most researchers.** With the highest keep rate (22.1%), lowest crash rate (8.7%), and best cost-effectiveness ($0.45 per 1% improvement), Sonnet 4.6 is the recommended starting point for autonomous optimization experiments.

3. **Extended thinking enables deeper optimization.** Opus 4.6 found multi-step optimization chains that other models could not, achieving the best absolute val_bpb (0.9019) on an already-optimized baseline. The cost premium is justified when the baseline is already strong and incremental improvements require causal reasoning about optimizer dynamics.

4. **Open-source tooling makes this accessible.** The `autoresearch` framework, combined with consumer-grade GPUs and standard API access, enables any researcher to run autonomous optimization experiments. The total cost for a comprehensive 100-experiment run with Sonnet 4.6 is under $11 including GPU electricity.

Phase 2 of this study (Section 5), currently in progress, directly addresses the two primary limitations: baseline inconsistency and single-run variance. Preliminary Phase 2 results for GPT-4.1 are being collected at time of writing. Additional future work should explore multi-GPU scaling, architecture search with more flexible compilation frameworks, evaluation on downstream tasks rather than proxy metrics, and cross-hardware validation with collaborator-contributed results.

These results suggest that LLMs are becoming practically useful for the specific subtask of iterative training optimization. Within a well-defined optimization loop with clear metrics, they can discover meaningful improvements autonomously. This study does not evaluate broader research tasks such as question formulation or scientific contextualization, and the degree to which these results generalize beyond hyperparameter tuning remains an open question.

Acknowledgments

We thank Andrej Karpathy for creating the `autoresearch` framework and for the broader vision of autonomous ML research that inspired this work.

We thank the open-source community whose tools made this work possible: the PyTorch team for `torch.compile` and the broader framework; the Triton team for GPU kernel compilation; Hugging Face for dataset hosting and the Transformers ecosystem; and Will McGugan and the Textualize team for the Rich TUI library used in experiment monitoring.

We thank Anthropic for API access to Claude Sonnet 4, Claude Sonnet 4.6, and Claude Opus 4.6, and for technical support during the extended thinking integration.

We thank Microsoft Azure and OpenAI for GPT-4.1 API access.

We gratefully acknowledge Renato Umeton, Ph.D., for guidance on publication and for championing open-source contributions to the ML research community.

We thank Dave Graham from ML Commons for working alongside us, sharing insights from his parallel research efforts, and for the accountability and motivation that made the overnight experiment runs possible.

We thank the LinkedIn ML/NLP community for feedback, discussion, and interest in early results shared during the course of this research.

References

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Pinto de Oliveira, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry,

P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

T. Dao. FlashAttention-3: Fast and exact attention with asymmetric attention. *arXiv preprint*, 2024.

F. Hutter, L. Kotthoff, and J. Vanschoren, editors. *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019.

A. Karpathy. autoresearch. https://github.com/karpathy/autoresearch, 2026.

C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

OpenAI. GPT-4.1. https://platform.openai.com, 2025.

Anthropic. Claude Sonnet 4.6 and Opus 4.6. https://www.anthropic.com, 2026.

PyTorch Team. `torch.compile`: Making PyTorch code faster. https://pytorch.org/docs/stable/torch.compiler.html, 2024.

K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. https://kellerjordan.github.io/posts/muon/, 2024.

B. Zoph and Q. V. Le. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2017.